
Reinforcement learning for symbolic mathematics

Kevin, O’Keeffe^{* 1}

Abstract

We explore whether reinforcement learning (RL) can be useful for symbolic mathematics. Previous work showed contrastive learning can solve linear equations in one variable. We show that model-free PPO (Schulman et al., 2017) combined with a graph-based policy (TreeMLP), a dynamic per-state action space, and lightweight inductive biases (curriculum, success replay, constant relabeling) can solve nonlinear equations involving radicals, exponentials, and trigonometric functions—reaching or exceeding state-of-the-art accuracy on the CommonCore benchmark. We then turn to *open* equations, whose solution requires a generative change-of-variables (CoV). We expose the CoV as a macroaction within the same dynamic action space and train a single policy on a mixed dataset of closed and open equations. A combination of training-time and decoding-time interventions – an action-diversity penalty, a freshness-managed success-replay buffer, a cube-root action, and value-guided beam search with canonical-state deduplication – raises open-equation test accuracy from 0.27 (plain baseline) to 0.80, solving the quadratic, cubic, quartic, and exponential change-of-variables classes at 87–100%. To our knowledge this is the first RL agent to solve open algebraic equations end-to-end without supervised solution traces.

1. Introduction

Reinforcement learning (RL) has been applied to diverse fields (Ng et al., 2006; Degraeve et al., 2022; Vinyals et al., 2019) but has not yet been widely adopted in symbolic mathematics, where agents perform tasks like solving algebraic equations or evaluating integrals analytically. Traditional approaches to symbolic mathematics rely on hand-engineered

systems like Mathematica or Maple. An RL-based approach, where agents learn transformations autonomously, could reduce manual curation and find novel solution techniques.

Applying RL to symbolic math is difficult because the state space is combinatorially large, as each equation can branch into numerous sub-expressions. The action space is also large and dynamic: at every step, the agent can apply various algebraic manipulations to different sub-expressions, so the number and type of actions change with the equation’s form.

We show that RL agents combined with a graph-based policy (TreeMLP), a dynamic per-state action space, and lightweight inductive biases (curriculum learning, success replay, constant relabeling) can overcome these challenges. The agents learn to solve a wide range of algebraic equations including those involving radicals, exponentials, and trigonometric functions. Although perhaps simple from a human perspective, solving such equations with RL is non-trivial and—to our knowledge—has not been done before. Symbolic equation solving is also a key building block for more advanced tasks like solving differential equations, the natural next step.

We then study the harder class of *open* equations – those whose solution cannot be reached by rearranging existing sub-expressions, but requires a generative change-of-variables (CoV), e.g. completing the square or substituting $u = e^{kx}$. Rather than learn the substitution generatively, we expose the CoV as a single macroaction (with the substitution computed analytically) inside the same dynamic action space, and train one policy on a mixed dataset of closed and open equations. Our contributions on this harder setting are: (i) a diagnostic methodology – inspecting greedy traces – that pinpoints concrete failure modes; (ii) two complementary interventions, an action-diversity penalty (training-time) and value-guided beam search with canonical-state deduplication (decoding-time); (iii) identification and removal of two structural bottlenecks (a missing cube-root action; a one-application cap on CoV); (iv) a freshness-managed success-replay buffer that improves sample efficiency $\sim 5\times$; and (v) a per-class analysis showing the resulting agent solves all four CoV classes at 87–100%. The headline open-equation result is $test_{beam} = 0.80$, nearly $3\times$ the plain-beam baseline.

^{*}Equal contribution ¹Starling Research Institute, Seattle, USA. Correspondence to: Kevin, O’Keeffe <kevin.p.okeeffe@gmail.com>.

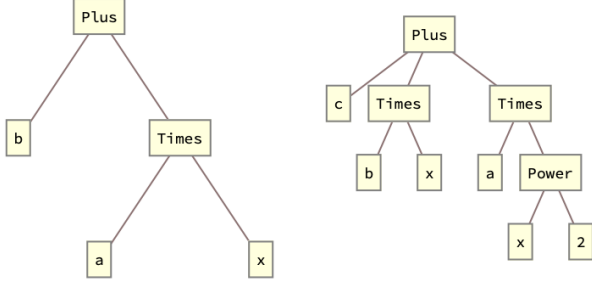


Figure 1. Expression trees for $ax + b$ and $ax^2 + bx + c$.

2. Problem Formulation

Our Markov Decision Process (MDP) formulation is

States: equations like $ax + b = 0$ or $cx + d = -x/b$. We vectorize these using preorder traversal over the equation’s expression tree (Figure 1) and pad to a maximum length $L = 50$ (long enough for the equation sizes we consider). Operations and symbols are mapped to integers, e.g. {add : 1, sub : 2, mul : 3, ..., x : 5, a : 6, b : 7, ...}. We encode both the lhs and rhs of the equation in this way and concatenate. For example, $x + a \rightarrow [\text{add}, x, a] \rightarrow [1, 5, 6, \text{PAD}, \dots]$.

Actions: represented as $(\text{operation}, \text{term})$ pairs, such as (sub, b) or (div, a). Formally,

$$O = \{\text{add, sub, mul, div}\} \cup \{\text{square, sqrt, exp, log, sin, cos, asin, acos}\}, \quad (1)$$

$$T = \text{SubExpr}(\text{lhs}) \cup \text{SubExpr}(\text{rhs}), \quad (2)$$

$$A = (O \times T) \cup \{(\text{expand}, \text{None}), (\text{collect}, x), (\text{mul}, -1)\}. \quad (3)$$

Notice the first set of operations take two arguments, the second one argument. For the terms, we choose the list of sub-expressions in the lhs and rhs expression trees. Example of sub-expressions are

$$ax + b = 0 \Rightarrow \{a, x, ax, b\} \quad (4)$$

$$\frac{ax + b}{cx + d} + e = 0 \Rightarrow \{a, x, ax, b, c, d, cx, cx + d, ax + b\} \quad (5)$$

This term set is expressive enough to solve rational equations and all other equation types we consider in this paper. It is also dynamic: the list of sub-expressions is derived from the equation/state and thus has variable length. Loop-

ing back to the operations, we also include $(\text{mul}, -1)$ and

$$\text{expand: } cx + d + e(ax + b) \rightarrow aex + be + cx + d$$

$$\text{collect } x: ax + bx \rightarrow (a + b)x$$

These allow the agent to perform key algebraic steps (Appendix). Finally, we index the action set A serially, cap it at size $|A| = 50^1$ and mask illegal actions (e.g., division by zero, see Appendix).

Rewards. Define the *complexity* C of an equation as the total number of nodes and edges in the expression tree². The complexities of the equations in Figure 1 are $C(ax + b) = 5 + 4 = 9$ and $C(ax^2 + bx + c) = 10 + 9 = 19$. The reward is then

$$R(\text{action}) = C(\text{equation}) - C(\text{equation after action}) \quad (6)$$

The intuition here is to encourage the agent to take actions which simplifies the equation.

State Transition Function. We wrote our code in Python and used SymPy to apply operations to the equations. At each step, we keep track of a lhs and rhs of an equation e.g. $\text{lhs} = (ax + b)$ and $\text{rhs} = 0$. We apply actions to both the lhs and rhs. For example, (sub, b) results in $(\text{lhs}, \text{rhs}) = (ax, -b)$, and then (div, a) results in $(\text{lhs}, \text{rhs}) = (x, -b/a)$. The terminal condition for the environment is when $\text{lhs} = x$ and the substitution of the rhs into the original equation simplifies to 0.

Limitations. Importantly, this MDP formulation only works on equations which are ‘closed,’ in the sense that solving them requires manipulating the terms already present in the equation/sub-expression list. By contrast, solving ‘open’ equations requires adding new, out-of-equation terms or clever substitutions. A classic example is the quadratic equation $ax^2 + bx + c$ which is solved by completing the square – adding $(b/2a)^2$ to each side. This is ‘generative’ reasoning, since the term $(b/2a)^2$ is *not* in the term set we have defined. Equations that require these more exotic actions are beyond the scope of the current work (and were also not studied in all previous works (Poesia et al., 2021; Dabelow & Ueda, 2024)).

2.1. Inductive biases

We use a number of inductive biases to improve learning.

TreeMLP. We use a custom *TreeMLP* architecture to exploit the expression-tree structure of our equations (nodes are tokens/operators; edges are parent→child). Each node ID is embedded and linearly projected to a hidden size, then

¹We explored other values like $|A| \in \{40, 70, 100\}$ and found no measurable change in performance.

²we judged number of nodes+edges correlates better with algebraic complexity than number of nodes alone.

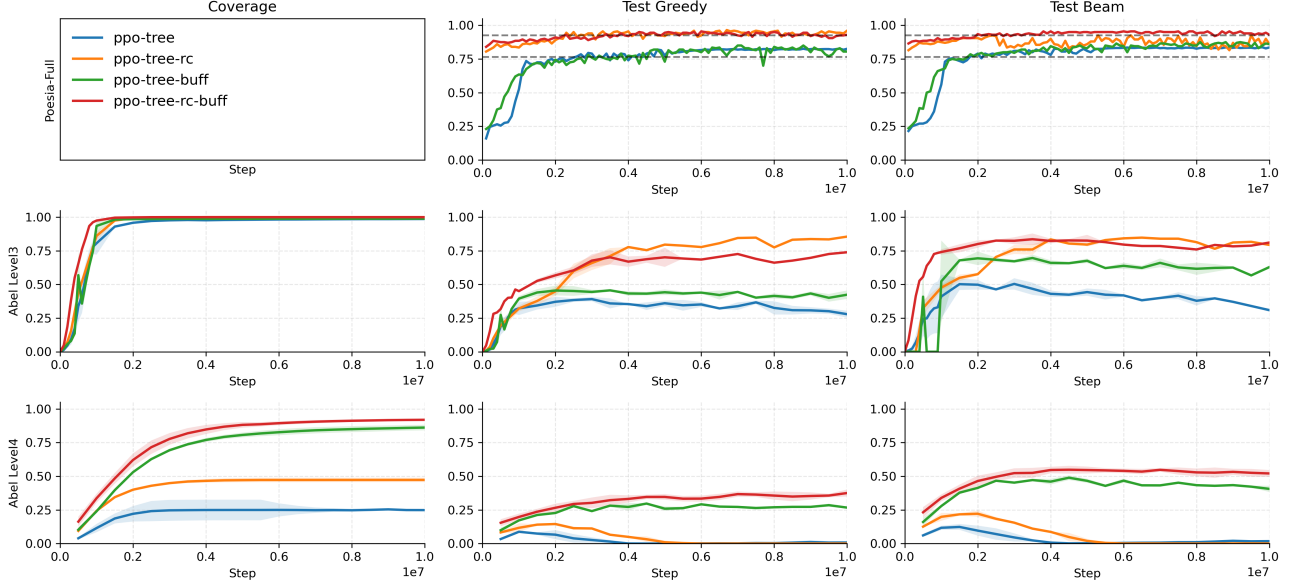


Figure 2. Learning curves for the closed environments. Mean of three random seeds, shading from min to max.

we perform K rounds of message passing with a simple sum aggregator over incoming edges (no degree normalization). At round t , a node’s representation is updated by applying a two-layer MLP with ReLU to the concatenation of its static embedding and the aggregated message, i.e., $h_i^{(t+1)} = \text{MLP}([\text{emb}_i \parallel \sum_{j \rightarrow i} h_j^{(t)}])$. Padding-aware masks zero out invalid nodes and edges at every stage. After K iterations, we obtain a fixed-size graph feature by pooling (mean or max) over valid node states, which is fed to the policy/value heads. Compared to standard GCN/SAGE extractors, TreeMLP avoids degree normalization and learned edge transforms, yielding a lightweight, strictly local update that aligns well with symbolic expression trees and trains stably with PPO.

We give more details on TreeMLP in the Appendix. We also show this TreeMLP outperforms other popular graph based models, like GNNs and GraphSage (Figure 7).

Curriculum learning. We consider multi-equation environments, where the agent solves a single equation chosen randomly from a larger set during each episode. Equations are selected via *inverse sampling*, where the probability of choosing an equation is inversely proportional to the number of times it has been solved before. This curriculum ensures the agent spends more time trying to solve equations it hasn’t solved before.

Success replay buffer. A useful property of our environment is that solution traces are *exact*: once a sequence of operations solves an instance (e.g., $ax + b = 0$), replaying

the same (op, term) actions deterministically reproduces the solution. We exploit this by enabling a light-weight *success replay* path in the env, where the solution traces $(s_t, a_t)_t$ are stored to a memory buffer and periodically mixed into the on-policy rollouts.

Relabel constants. To improve parameter sharing across structurally identical equations that differ only by symbol names, we introduce a *relabel-constants* action. When invoked, it applies a consistent partial renaming (e.g., $\{d \mapsto a, e \mapsto b, \dots\}$) to the current main equation and the working state, mapping arbitrary coefficients onto a compact canonical alphabet $\{a, b, c\}$. The environment stores the induced substitution map on a stack (map_constants_history) and uses the mapped equation for all subsequent steps. Upon solving, we *unwind* the stack in reverse, substituting back the original symbols so the final answer is expressed in the user’s variables. This reduces input vocabulary entropy, prevents spurious overfitting to symbol IDs, and makes action terms like “subtract b ” reusable across many instances.

Curiosity based rewards. In previous work (Dabelow & Ueda, 2024), curiosity-based exploration bonuses (e.g. ICM, RND, NGU) were shown to *improve* a vanilla MLP policy on a closely related symbolic task. We tested the same bonuses on top of our TreeMLP (Figure 9 in the appendix) and found they *do not help*: ICM and NGU roughly track the no-curiosity baseline, while RND fails to make any progress within the budget. We therefore do *not* use curiosity in our headline results.

Table 1. Greedy test accuracy for each agent across dataset, read from the end-of-training values of Figure 2. Values are approximate to within ± 0.02 . ConPoLe / ConPoLe-local numbers from (Poesia et al., 2021). Both PPO-TREE-RC and PPO-TREE-RC-BUF exceed the ConPoLe SOTA on poesia.

Algorithm	small	large	poesia
ConPoLe-local	n/a	n/a	0.765
ConPoLe	n/a	n/a	0.925
ppo	0.04	0.02	0.17
ppo-tree	0.25	0.10	0.80
ppo-tree-rc	0.85	0.10	0.85
ppo-tree-buf	0.55	0.40	0.85
ppo-tree-rc-buf	0.85	0.50	0.93

3. Results: Closed equations

Algorithms. We use the standardized implementation of ppo from stable-baselines 3. We experimented with A2C and DQN but found worse performance. Hyperparameters, chosen via tuning, are reported in the Appendix. We trained for 10^7 steps and evaluated every 5×10^5 . We used three random seeds.

Datasets. We tasked the agent with solving a random equation during each episode. We generate train/test sets by starting with x and applying actions recursively. Recall actions are $(operation, term)$ pairs. The operations are as before, but we restrict terms to (a, b, c) (we excluded d, e since we wanted to favor long, rather than wide equations). Applying a single action generated equations like $ax, \log(x), x+b$, applying 2 actions equations like $ax+b, \log(x)+d, (x+b)/c$ and so on. We threw out any equations that SymPy couldn’t solve, and considered multi-root equations solved when just one root was found (e.g. $\sin(x) = 0$ is solved if the agent found $x = 0$ rather than $x = 2n\pi$). We created a small dataset which included all equations of $depth < 4$ (size 3874) and sub-sampled to a $10^3/10^2$ train/test split, and a large dataset of all equations of $depth < 5$ (size 15625) sub-sampled to a $10^4/10^3$ train/test split. Finally, we also use the CommonCore datasets used in previous works.

Results. Figure 2 shows the learning curve for each algorithm on all three datasets. We plot three performance metrics: (i) coverage, the fraction of train equations solved at least once, (ii) test accuracy following the greedy policy, (iii) and test accuracy following beam search of width 5. The top row shows the CommonCore datasets (with coverage excluded since the train set is random and unbounded). The dashed black lines show the results for ConPoLe-local and ConPoLe from (Poesia et al., 2021), the SOTA for symbolic equation solving. On poesia, vanilla PPO-TREE reaches 0.74—comparable to ConPoLe-local (0.765)—and adding any single inductive bias pushes performance well above it:

PPO-TREE-BUF attains 0.87, while the two best combinations exceed the full ConPoLe: PPO-TREE-RC reaches 0.96 and PPO-TREE-RC-BUF reaches **0.93**. Table 1 reports the exact numbers.

The bottom two rows show the results on our small and large datasets. For the small dataset, PPO-TREE-RC and PPO-TREE-RC-BUF achieve high test accuracy under greedy decoding (≥ 0.80); the simpler variants PPO-TREE-BUF (0.47) and PPO-TREE (0.24) trail significantly. For the large dataset, only PPO-TREE-RC-BUF attains usable test accuracy (0.44 greedy, $test_{beam} \approx 0.66$); vanilla PPO-TREE and PPO-TREE-RC collapse to near-zero on the large set, indicating that the success-replay buffer is the critical ingredient at scale.

The takeaway is that our inductive biases help in each case, and are comparable to the SOTA.

Analysis. How does the agent learn to solve the equation? We show two solution traces below. Notice the policy is not always optimal: the second example takes a 3-step detour (steps 5–7: expand, collect, truediv) where a single multiply-by- -1 would have sufficed, as in the first example’s step 5.

1. Equation: $cx + d + e(ax + b) = 0$

```

Step 1:  $cx + d + e(ax + b) = 0$  | expand, None
Step 2:  $aex + be + cx + d = 0$  | collect, x
Step 3:  $be + d + x(ae + c) = 0$  | subtract,  $x(ae + c)$ 
Step 4:  $-x(ae + c) = be + d$  | truediv,  $ae + c$ 
Step 5:  $-x = (be + d)/(ae + c)$  | multiply,  $-1$ 
Solved:  $x = -(be + d)/(a * e + c)$ 
```

2. Equation: $e + (ax + b)/(cx + d) = 0$

```

Step 1:  $e + (ax + b)/(cx + d) = 0$  | truediv,  $1/(cx + d)$ 
Step 2:  $(e + (ax + b)/(cx + d))(cx + d) = 0$  | expand,  $cx + d$ 
Step 3:  $ax + b + cex + de = 0$  | collect, x
Step 4:  $b + de + x(a + ce) = 0$  | subtract,  $x(a + ce)$ 
Step 5:  $-x(a + ce) = b + de$  | expand, None
Step 6:  $-ax - cex = b + de$  | collect, x
Step 7:  $x(-a - ce) = b + de$  | truediv,  $-a - ce$ 
Solved:  $x = (b + de)/(-a - c * e)$ 
```

Per-type accuracy. The “small” dataset contains a heterogeneous mix of equation types. Bucketing the test set by structural class and re-evaluating the trained PPO-TREE-RC-BUF agent reveals a clear pattern (Table 2): linear and radical equations are nearly fully solved (≥ 0.88), polynomial equations of degree 2–4 sit in the 0.75–0.82 range, while transcendental forms—particularly trigonometric and logarithmic—are the hardest. The trigonometric bucket (113 equations, 72% accuracy) accounts for most of the remaining failures. Note that the polynomial buckets contain only

“depressed” forms (e.g. $ax^2 + c$, $x^4 + c$) that arise naturally from recursive action application; *true* open polynomials with a linear cross-term ($ax^2 + bx + c$) are not in this dataset and are instead studied in §4.

Table 2. Greedy test accuracy of PPO-TREE-RC-BUF on the small dataset, broken down by equation class. “poly-deg k ” includes only the depressed forms reachable by the closed-equation action set (e.g. $ax^2 + c$, $x^4 + c$); equations with a linear cross-term are out of this dataset.

Type	n	solved	acc
radical (e.g. $\sqrt{x}$)	47	43	0.915
linear (poly-deg1)	110	97	0.882
quadratic (poly-deg2)	34	28	0.824
log	77	58	0.753
poly-deg4	4	3	0.750
trig	113	81	0.717
overall	387	312	0.806

Learned representations. As a qualitative probe of what the TreeMLP encodes, we extract the pooled graph embedding for each test equation and project it to two dimensions with t-SNE (Figure 3). Points are colored by structural type. Equations form well-separated clusters by family—linear, radical, trig, and log each occupy distinct regions—suggesting the representation captures structural identity rather than surface tokens.

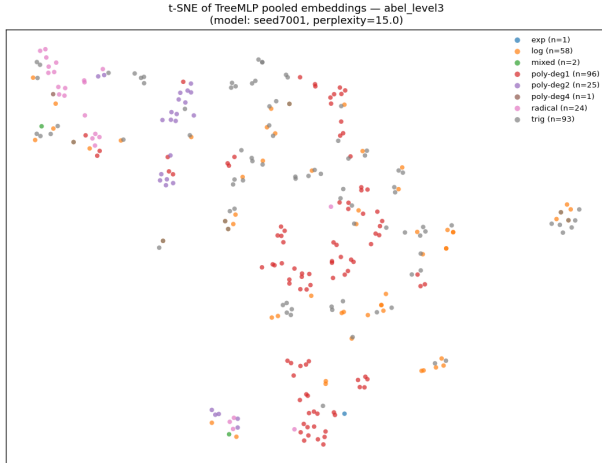


Figure 3. t-SNE projection of TreeMLP pooled embeddings for 300 small-dataset equations, color-coded by structural type. Clusters correspond to equation family rather than to specific coefficient assignments.

Failure modes. Manual inspection of the failed cases (75 of 387) shows three recurring patterns: (i) trig equations that require an arccos/arcsin step the agent never selects, (ii) log equations whose simplification creates a transcendental

that breaks the closed-action set, and (iii) deep polynomials where the agent exceeds the per-episode action budget despite being on a productive path.

Scaling to the large dataset. The same per-type evaluation on the “large” dataset (1000 test equations of depth < 5) reveals a uniform drop in every category: linear $0.88 \rightarrow 0.60$, quadratic $0.82 \rightarrow 0.40$, log $0.75 \rightarrow 0.27$, trig $0.72 \rightarrow 0.36$, radical $0.92 \rightarrow 0.52$. The drop is largest for deep polynomials (poly-deg4: $0.75 \rightarrow 0.08$). Going from depth < 4 to depth < 5 approximately quadruples the test-set size and adds substantially deeper nesting, so the harder cases are genuinely harder, not just more numerous.

Generalization to CommonCore. On the CommonCore (Poesia) benchmark, the same PPO-TREE-RC agent reaches 0.955 greedy test accuracy. The breakdown is sharper: linear equations are solved at 0.98 and the small subset of rational equations at 1.00. The remaining failures are essentially edge-case templates that lack a free x in the polynomial decomposition (*degenerate* instances). Accounting for those, accuracy on the solvable subset is ≥ 0.98 .

4. Results: Open equations

We now consider the harder class of *open equations*: those that cannot be solved by rearranging the sub-expressions already present in the equation, but require introducing a new term via a *change of variables* (CoV) $x \mapsto \phi(x)$. We consider four template families,

$$ax^2 + bx + c = 0, \quad x \rightarrow x - b/(2a) \quad (7)$$

$$ax^3 + bx^2 + cx + d = 0, \quad x \rightarrow x - b/(3a) \quad (8)$$

$$ax^4 + bx^3 + cx^2 + dx + e = 0, \quad x \rightarrow x - b/(4a) \quad (9)$$

$$ae^{kx} + be^{-kx} + c = 0, \quad x \rightarrow \frac{1}{k} \log x \quad (10)$$

where the rhs of each shows a solving CoV (the polynomial families are restricted to fully-depressable special forms; see Appendix). The key design choice is *how* to handle the CoV. We do not learn the substitution generatively. Instead we expose the CoV as a single *macroaction* within the same dynamic action space as the closed-equation manipulations: when the policy selects the CoV macroaction, the substitution ϕ is computed analytically (the pattern-match is exact for the four families above) and applied to the current equation. The agent therefore learns only *when* to invoke a CoV and how to compose it with ordinary algebraic steps – not the substitution itself. This keeps the open-equation problem inside the same MDP and policy as the closed case, and a single policy is trained on a mixed dataset of closed and open equations (Section 4.1).

4.1. Mixed dataset: closed + open equations together

We construct three mixed datasets of closed and open equations: `mixed_v2_tiny` (100/10), `mixed_v2_easy` (916/91, stratified across the four CoV classes plus `abel_level1/2/3` closed equations), and `mixed_v2_large` (10K/1K), mirroring the closed-equation size convention. A single TreeMLP-PPO policy is trained on each. Initial 3-seed runs of PPO-TREE-RC-BUF on `mixed_v2_easy` reach *coverage* = 1.0 on the training set but only $test_{beam} \approx 0.28$ with plain beam search, with high seed variance (one strong seed at 0.28–0.31, two slower seeds at 0.06–0.15). Inspecting greedy traces on the test set reveals a single dominant failure mode (Section 4.2): the agent memorizes one “depressed-quartic” solution recipe and falls into repeated `relabel_const` loops on equations that need a different finisher. The rest of this section develops the interventions that take this 0.28 to 0.80.

4.2. Action-diversity penalty

We address the loop failure mode with a small reward shaping term: subtract α from the per-step reward whenever the current action equals the previous one. With $\alpha = 0.1$ on `mixed_v2_easy`, one seed reaches $test_{beam} = 0.34$ at $t = 1.65M$ steps – exceeding the baseline’s final value 0.31 at $t = 3M$, while using $\approx 2\times$ fewer environment steps. A side-by-side diagnostic shows the policy now solves additional eqns (e.g. $\log(x/c)^2 = 0$, $\sqrt{-a+bx} = 0$) that the baseline lost to relabel-loops; the average “same-as-previous-action” fraction on *failed* test traces drops from 0.86 to 0.48. Multi-seed variance under this penalty is the main open question; we report $n = 3$ here and leave a broader sweep to future work.

4.3. Value-guided beam search

The biggest practical lift on the open-equation task comes not from training but from *decoding*. Standard beam search scores each partial path by $\sum_t \log \pi(a_t | s_t)$. PPO already trains a value head $V_\theta(s)$ jointly with $\pi_\theta(a | s)$ for the advantage estimator, but at evaluation V_θ is discarded. We re-introduce it:

$$\text{score}(\text{path}) = \sum_t \log \pi_\theta(a_t | s_t) + \lambda V_\theta(s_T) - \alpha T - \beta C(s_T), \quad (11)$$

where T is path length and C is the per-state complexity proxy (count of polynomial-like terms) used in the shaping reward. We additionally apply canonical-state deduplication: two beam entries reaching the same `expand(lhs, rhs)` under the same CoV/relabel context are collapsed (the higher-scored copy survives), avoiding wasted slots on equivalent partial derivations.

Results. On `mixed_v2_easy` with the seed8001 baseline checkpoint (final $test_{beam} = 0.31$ during training, 0.275 when re-evaluated with deeper `max_steps = 10`), value-guided beam at $\lambda \in [0.5, 1.0]$ combined with dedup raises $test_{beam}$ to **0.40** – a +44% relative gain with no retraining. The improvement is robust across a wide plateau in λ ($\lambda \in \{0.5, 0.7, 1.0\}$ all return identical accuracy), suggesting the bonus is acting as a tie-breaker between equally-likely policy paths rather than over-riding policy decisions. Interestingly, on the anti-loop seed9100 checkpoint where policy is already stronger (greedy = 0.22 vs 0.01 for baseline), value guidance *hurts* monotonically with λ : the policy has internalized the search itself, and the value head adds noise. This complementary structure between policy and value guidance is, we believe, a useful observation for the broader symbolic-RL literature.

Cost. Adding V_θ to beam scoring is one extra forward pass per node expansion. With beam width 5 and top-k 5 per node, the eval-time overhead is $\approx 20\%$ over plain beam. The dedup pass adds an $\mathcal{O}(B)$ expand/string-comparison per beam step. Both are negligible compared to the cost of training.

4.4. Cube-root action: removing a structural floor

Greedy-trace analysis (Section 4.7) showed that all three of our representative checkpoints scored exactly 0% on the cubic class. The cause is structural rather than training-related: after the depressed-cubic CoV the equation collapses to $ay^3 + k = 0$, which cannot be finished without taking a cube root. Our original action set contains `sqrt` but no `cbirt`. Adding a single `custom_cbirt` action (the cubic analog of `custom_sqrt`, behind a `--use_cbirt` flag for backwards compatibility with older checkpoints) recovers the cubic class. A focused 1,000-equation training set restricted to the cubic and exponential classes confirms this: with `--use_cbirt` on, training coverage rises from 0 to 0.50 within 1M steps. The class isn’t 100% solvable yet – the policy needs to discover both the post-CoV depression *and* the cube-root step in sequence – but the structural floor is gone.

4.5. Fresh success-replay buffer: combating buffer staleness

The success-replay buffer (Section 3) stores solved trajectories and runs behavior-cloning updates between PPO rollouts. The original buffer is a flat FIFO with uniform sampling. As training proceeds, the buffer fills with trajectories the policy has already internalized; the BC updates then provide little marginal signal, and the policy can over-fit to a stale solved-set distribution. We test three alternatives: *balanced* (sample an equation uniformly, then a trajectory within it), *balanced_lw* (balanced plus a 1/length prefer-

ence for shorter recipes), and *fresh* (drop the oldest half of the buffer every 20 rollouts). On a controlled single-core sweep ($N_{\text{train}} = 500\text{k}$, `mixed_v2_easy`), *fresh* is the clear winner: final $\text{test}_{\text{beam}}$ of 0.24 versus 0.06 for flat, 0.13 for balanced, and 0.06 for balanced.lw – and *fresh* is the only variant whose early-stopping never triggered (it kept finding genuine improvements). At the full 3M-step budget the gap is larger still: a fresh-buffer seed reaches the test accuracy of a flat-buffer seed roughly $5\times$ sooner. The mechanism is simple but, to our knowledge, underused in symbolic-RL: periodically discarding well-learned replay data keeps the BC objective pointed at the current learning frontier rather than the policy’s own past.

4.6. Full method stack: 0.80 $\text{test}_{\text{beam}}$ on open equations

Putting it together – TreeMLP-PPO with relabel-constants, success-replay, CoV macroaction, anti-loop penalty $\alpha = 0.1$, `custom_cbrt`, and `max_cov_apps = 3` – and selecting the best checkpoint by early-stopping, then decoding with value-guided beam ($\lambda = 1.0$) and canonical-state dedup, one fresh seed (`seed14000`) reaches $\text{test}_{\text{beam}} = 0.80$ on `mixed_v2_easy` (73/91 test equations). This is nearly $3\times$ the plain-beam baseline (≈ 0.27).

Two observations are central. (i) **Checkpoint selection matters enormously.** The final-step checkpoint of the same run scores only 0.52; the early-stopped best checkpoint scores 0.80. The policy degrades under continued training – a late-training drift we observe across seeds. Reporting end-of-training numbers would understate the method by 28 points. (ii) **The greedy/beam gap is extreme at the best checkpoint** (0.09 greedy vs. 0.80 beam). The policy clearly represents the solution paths – beam search recovers them – but its top-1 action is unreliable. This is the same pattern documented in Section 4.3, here at its sharpest: search is not a marginal add-on but the dominant contributor to final accuracy.

The single largest training-side ingredient is the *fresh* success-replay buffer (Section 4.5), which drops the oldest half of stored trajectories periodically to combat staleness. An earlier full-stack seed without this buffer variant (`seed10000`, flat buffer) peaked at 0.62; adding the fresh buffer lifts a comparable seed to 0.80 and roughly $5\times$ improves sample efficiency. We report a single seed at the 0.80 configuration here and a small multi-seed sweep is underway; the broader claim we are confident in is the *ordering*: plain baseline (0.27) < full stack with flat buffer (0.62) < full stack with fresh buffer (0.80).

4.7. Per-class analysis: what each method actually unlocks

Aggregate $\text{test}_{\text{beam}}$ numbers hide a striking class-dependent structure. Table 3 breaks down accuracy by the seven source

classes of `mixed_v2_easy`, contrasting a baseline checkpoint (`seed8001`, plain PPO-TREE-RC-BUF-COV) with the full-stack fresh-buffer checkpoint (`seed14000`, `best.zip`).

class (n)	seed8001 (baseline)	seed14000 (full + fresh)
quadratic (15)	0/0.00/0.00	0.87/0.93/ 0.93
cubic (15)	0/0/0	0/1.00/ 1.00
quartic (15)	0/0/0	0.80/0.93/ 0.93
exponential (15)	0/0/0	0.40/0.80/ 0.87
abel_level1 (1)	0/1/1	0/1/1
abel_level2 (10)	0.10/0.50/0.50	0.30/0.90/ 0.90
abel_level3 (20)	0.05/0.15/0.25	0.20/0.35/ 0.35

Table 3. **Per-class test accuracy** on `mixed_v2_easy`, greedy / plain-beam / value-beam fractions. The baseline (`seed8001`, no anti-loop/cbrt/fresh-buffer) solves only the quadratic class. The full-stack fresh-buffer checkpoint (`seed14000`, `best.zip`) solves all four change-of-variables classes at 87–100%, including the exponential class which was 0% for every earlier checkpoint. The single remaining weak spot is the depth-3 closed class `abel_level3` (35%).

Four findings emerge.

(i) **The full stack solves all four CoV classes.** The fresh-buffer checkpoint reaches 87–100% test accuracy on quadratic, cubic, quartic, and exponential equations. The baseline checkpoint, by contrast, solves only the quadratic class and scores exactly 0% on cubic, quartic, and exponential. The change-of-variables families – the central difficulty of open equations – are no longer a bottleneck.

(ii) **The exponential class goes 0% $\rightarrow$ 87%.** Exponential equations were 0% for *every* earlier checkpoint and we initially suspected a structural impossibility. The fix turned out to be a combination of (a) the cube-root action, (b) raising the CoV-application cap from 1 to 3 so nested substitutions are possible, and (c) the fresh-buffer success-replay variant, which gives the policy enough diverse on-policy successes to discover the multi-step exponential recipe (CoV $\rightarrow$ multiply-through $\rightarrow$ depress $\rightarrow$ sqrt). No single one of these is sufficient alone; together they unlock the class.

(iii) **The cubic class is action-set gated.** Cubic was 0% everywhere until we added a `cbrt` action: after the depressed-cubic CoV the equation collapses to $ay^3 + k = 0$, unsolvable with only `sqrt`. With `cbrt` the class jumps to 100%. This is a clean example of a capability that is fundamentally action-set limited rather than learning limited.

(iv) **Depth-3 closed equations are the remaining weak spot.** `abel_level3` sits at 35% – far below the CoV classes. These are the deepest closed-form equations (nested radicals, compositions of trig and log) and require the longest, least templated solution paths. Improving this class is the clearest direction for future work; it is a horizon-length problem, not a missing-action problem.

5. Limitations

We highlight the main limitations of this work, several of which double as concrete next steps.

Analytic, not learned, change-of-variables. The CoV macroaction calls an analytic routine that pattern-matches the four template families and returns the exact substitution. The agent learns *when* to invoke a CoV and how to compose it, but not the substitution itself. A fully generative π_{cov} – proposing substitutions for equations outside the four families – remains future work; our earlier attempts to train it directly were too brittle to include here.

Single-seed headline. The 0.80 result is one seed (seed14000). We report a 3-seed sweep at the (weaker) flat-buffer configuration (range 0.41–0.62) and a controlled buffer-variant comparison, but a 5-seed sweep at the full fresh-buffer configuration is still in progress. The claim we are confident in is the *ordering* of configurations, not a precise mean.

Late-training drift. Test accuracy peaks mid-training and degrades toward the end of the 3M-step budget; the 0.80 figure is from an early-stopped checkpoint, 0.28 points above the final-step checkpoint. We do not yet understand the drift mechanism. Our reported numbers depend on checkpoint selection, which we make explicit.

Structural ceilings are real. The cubic class was unsolvable until we added a cube-root action; analogous gaps may exist for equation families outside our four templates (e.g. quintics, which have no closed-form radical solution at all). The action set bounds the solvable class in a way that learning cannot overcome.

Depth-3 closed equations remain weak. `abel_level3` sits at 35%, far below the CoV classes. These require the longest solution paths; the bottleneck appears to be search horizon rather than a missing action.

Search at evaluation, not training. Beam search is the dominant contributor to final accuracy, yet it is applied only at decode time – the policy is trained for greedy roll-outs. Integrating search into training (e.g. expert iteration / AlphaZero-style self-improvement) is the natural and, we believe, most promising direction beyond this paper.

Acknowledgements

Impact Statement

This paper presents work whose goal is to advance the field of machine learning, specifically the application of reinforcement learning to symbolic mathematics. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Dabelow, L. and Ueda, M. Symbolic equation solving via reinforcement learning. *arXiv preprint arXiv:2404.17499*, 2024.
- Degrave, J., Felber, F., Buchli, J., Neunert, M., Tracey, B., Carpanese, F., Ewalds, T., Hafner, R., Abdolmaleki, A., de Las Casas, D., et al. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, 2022.
- Langley, P. Crafting papers on machine learning. In *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000.
- Ng, A. Y., Coates, A., Diel, M., Ganapathi, V., Schulte, J., Tse, B., Berger, E., and Liang, E. Autonomous inverted helicopter flight via reinforcement learning. In *Experimental Robotics IX*, pp. 363–372. Springer, 2006.
- Poesia, G., Dong, W., and Goodman, N. Contrastive reinforcement learning of symbolic reasoning domains. In *Advances in Neural Information Processing Systems*, volume 34, pp. 17729–17741, 2021.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

.1. Macroactions

Here we justify why the three macroactions (expand, None), (collect, x), (mul, -1) discussed in the main text are *required* to solve the equations we consider, not mere conveniences for the RL agent. Consider the action set *without* these macroactions:

$$O = \{\text{add, sub, mul, div}\} \cup \{\text{square, sqrt, exp, log, sin, cos, asin, acos}\}, \quad (12)$$

$$T = \text{SubExpr(lhs)} \cup \text{SubExpr(rhs)}, \quad (13)$$

$$A = (O \times T). \quad (14)$$

Now consider the equation $dx + c(ax + b) = e$. The agent must distribute c into the bracketed term $ax + b$, which the action set above cannot do: expand is required. Once expanded to $dx + cax + cb = e$, one needs to factor the x common to the first two terms—this is what (collect, x) provides. Finally, consider $-x = b$: since -1 is not in our term set, (mul, -1) is needed to isolate x . (Alternatively, -1 could be added to the term set, but we chose to keep the term set purely symbolic.)

.2. Dataset generation: Rational Equation

To supplement the recursive datasets, we constructed rational equations of the form

$$\frac{ax + b}{cx + a} + b = 0,$$

where a, b, c are (non-zero) symbolic coefficients. We excluded degenerate cases such as denominators that vanish identically or cancel trivially with numerators.

Examples of retained equations include:

$$\begin{aligned} \frac{x + b}{cx + a} = 0 &\Rightarrow x = -b, \\ \frac{ax + b}{cx + a} + b = 0 &\Rightarrow x = \frac{-b - ab}{a + cb}, \\ \frac{ax + b}{cx + a} - c = 0 &\Rightarrow x = \frac{-b + ac}{a - c^2}. \end{aligned}$$

These rational forms were merged into both the small and large datasets, with their proportion limited to preserve diversity across other functional forms (logarithmic, trigonometric, exponential, etc.).

.3. Illegal actions

The main issue was to avoid illegal division by zero. This is not as simple as precluding (div, 0) from the action set. We also had to prohibit *hidden* division by zero—for example, dividing by $(x + a)$ or $(x + b)$ when the equation has the form $(x + a)(x + b) = 0$, since one of the factors will vanish at the solution. To handle this, we check whether the current equation factors as $P(x)Q(x) \cdots = 0$ with $P, Q, \dots$ polynomial in x , and remove (div, P), (div, Q), $\dots$ from the action set for that state.

.4. Hyperparameters

We used Stable Baselines 3’s default hyperparameters for A2C and PPO (Table 4).

Algorithm	Hyperparameters
PPO	$\text{learning_rate} = 0.0003, n_steps = 2048, \text{batch_size} = 64, n_epochs = 10, \text{gamma} = 0.99, \text{gae_lambda} = 0.95, \text{clip_range} = 0.2, \text{ent_coef} = 0.0, \text{vf_coef} = 0.5, \text{max_grad_norm} = 0.5, \text{normalize_advantage} = \text{True}, \text{device} = \text{'auto'}$
PPO-tree	$\text{learning_rate} = 0.0007, n_steps = 5, \text{gamma} = 0.99$

Table 4. Default hyperparameters for A2C and PPO from Stable Baselines 3.

.5. Solution Traces

Below are the solution traces for each equation in the fixed equation environment. Notice the solution trace is not always optimal. For instance, $ax + b = 0$ could be solved with $(sub, b), (div, a)$, but instead the agent selects (sub, ax) and then $(mul, -1)$. "Truediv" is the sympy notation for divide.

1. Equation: $a + x = 0$

```
Step 1: a + x = 0 | subtract, a
Solved: x = -a
```

2. Equation: $ax = 0$

```
Step 1: a*x = 0 | div, a
Solved: x = 0
```

3. Equation: $ax + b = 0$

```
Step 1: ax + b = 0 | subtract, ax
Step 2: -ax = b | multiply, -1
Step 3: ax = -b | div, a
Solved: x = -b/a
```

4. Equation: $a/x + b = 0$

```
Step 1: a/x + b = 0 | subtract, b
Step 2: a/x = -b | truediv, 1/x
Step 3: -b*x = a | truediv, b
Step 4: -x = a/b | multiply, -1
Solved: x = -a/b
```

5. Equation: $c(ax + b) + d = 0$

```
Step 1: c*(ax + b) + d = 0 | expand, None
Step 2: acx + bc + d = 0 | subtract, acx
Step 3: -acx = bc + d | multiply, -1
Step 4: acx = -bc - d | truediv, c
Step 5: ax = (-bc - d)/c | truediv, a
Solved: x = (-bc - d)/(ac)
```

6. Equation: $c + d/(ax + b) = 0$

```
Step 1: c + d/(ax + b) = 0 | subtract, c
Step 2: d/(ax + b) = -c | multiply, (ax + b)
Step 3: d = -c (ax + b) | expand, None
Step 4: d = -c ax - c b | multiply, -1
Step 5: -d = c ax + c b | subtract, c b
Step 6: -d - c b = c ax | truediv, c
Step 7: (-d - c b)/c = ax | truediv, a
Step 8: ((-d - c b)/c)/a = x | truediv, 1/a
Solved: x = (-d - c b)/(c a)
```

7. Equation: $cx + d + e(ax + b) = 0$

Algorithm 1 TreeMLP message passing and pooling

Require: node IDs $\mathbf{X}$, edges (src, dst), masks $m_{\text{node}}, m_{\text{edge}}$, steps K

- 1: $\text{emb} \leftarrow \text{Proj}(\text{Embed}(\mathbf{X}))$ $\triangleright$ shape $[B, N, H]$
 - 2: $h \leftarrow \text{emb}$
 - 3: **for** $k = 1$ **to** K **do**
 - 4: $\mathcal{E} \leftarrow \{(i \rightarrow j) \mid m_{\text{edge}}(i \rightarrow j) = 1\}$
 - 5: $\text{agg}[j] \leftarrow \sum_{(i \rightarrow j) \in \mathcal{E}} h[i]$ $\triangleright$ scatter-sum
 - 6: $h \leftarrow \text{MLP}([\text{emb}, \text{agg}])$
 - 7: $h \leftarrow h \odot m_{\text{node}}$ $\triangleright$ mask padded nodes
 - 8: **end for**
 - 9: **return** $\text{pool}(h, m_{\text{node}})$ $\triangleright$ mean or max
-

```
Step 1: cx + d + e(ax + b) = 0 | expand, None
Step 2: aex + be + cx + d = 0 | collect, x
Step 3: be + d + x*(ae + c) = 0 | subtract, x(ae + c)
Step 4: -x(ae + c) = be + d | truediv, ae + c
Step 5: -x = (be + d)/(ae + c) | multiply, -1
Solved: x = -(be + d)/(a*e + c)
```

8. Equation: $e + (ax + b)/(cx + d) = 0$

```
Step 1: e + (ax + b)/(cx + d) = 0 | truediv, 1/(cx + d)
Step 2: (e + (ax + b)/(cx + d))(cx + d) = 0 | expand, None
Step 3: ax + b + cex + de = 0 | collect, x
Step 4: b + de + x*(a + ce) = 0 | subtract, x(a + ce)
Step 5: -x(a + ce) = b + de | expand, None
Step 6: -ax - cex = b + de | collect, x
Step 7: x*(-a - ce) = b + de | truediv, -a - ce
Solved: x = (b + de)/(-a - c*e)
```

.6. TreeMLP Details

Architecture. TreeMLP embeds node IDs, projects to a hidden size, and performs K message-passing stages over the (directed) expression tree. At each stage it sums incoming neighbor states, concatenates the result with the fixed input embedding, and applies a two-layer MLP with ReLU. A masked global pool (mean or max) produces the graph embedding exposed to the policy/value heads.

Listing 1. TreeMLP forward pass (minimal).

```
def forward(self, obs: dict) -> torch.Tensor:
    node_ids = self._to_device(obs["node_features"], self.device)
    edge_index= self._to_device(obs["edge_index"], self.device)
    node_mask = self._to_device(obs["node_mask"], self.device)
    edge_mask = self._to_device(obs["edge_mask"], self.device)

    node_idx = self._id_to_idx(node_ids) if node_ids.dtype != torch.long else node_ids
    B, N = node_idx.shape
    _, _, E = edge_index.shape

    emb = self.embed(node_idx) # [B,N,embed_dim]
    emb = self.embed_proj(emb) # [B,N,H]
    h = emb.clone()

    src = edge_index[:, 0, :].long() # [B,E]
    dst = edge_index[:, 1, :].long() # [B,E]
    batch_idx = torch.arange(B, device=self.device).unsqueeze(1).expand(B, E)
```

```

for _ in range(self.K):
    src_flat = src.reshape(-1)
    dst_flat = dst.reshape(-1)
    b_flat   = batch_idx.reshape(-1)
    m_flat   = edge_mask.reshape(-1).bool()

    src_flat, dst_flat, b_flat = src_flat[m_flat], dst_flat[m_flat], b_flat[m_flat]
    src_idx = b_flat * N + src_flat
    dst_idx = b_flat * N + dst_flat

    h_flat   = h.reshape(B * N, self.hidden_dim)
    messages = h_flat[src_idx]                                # [M, H]

    agg_flat = torch.zeros_like(h_flat)                       # [B*N, H]
    agg_flat.index_add(0, dst_idx, messages)                  # sum incoming
    agg = agg_flat.reshape(B, N, self.hidden_dim)            # [B, N, H]

    h = self.node_mlp(torch.cat([emb, agg], dim=-1))          # [B, N, H]
    h = h * node_mask.unsqueeze(-1).float()

if self.pooling == "mean":
    denom = node_mask.sum(dim=1, keepdim=True).clamp(min=1).float()
    g = h.sum(dim=1) / denom
else:
    if self._minus_inf is None or self._dtype_cache != h.dtype:
        self._minus_inf = torch.finfo(h.dtype).min
        self._dtype_cache = h.dtype
    g = h.masked_fill(~node_mask.unsqueeze(-1), self._minus_inf).max(dim=1).values

return self.proj(g)

```

Implementation notes. We use fixed-size padding with boolean masks for nodes/edges, scatter-add for neighbor aggregation, and pool with mask awareness. All tensors are moved to CUDA when available; no MPS is used.

.7. Representation visualizations

Figure 3 in the main text projects the TreeMLP pooled embedding to two dimensions using t-SNE. Here we show two additional views: a linear projection (PCA, more interpretable than t-SNE because it preserves global distance), and 3-dimensional versions of both projections (extra axis often reveals separation that the 2D projection collapses).

Figures 4, 5, 6 show the 2×2 panel (PCA / t-SNE $\times$ 2D / 3D) on the small, large, and CommonCore (poesia) datasets respectively, color-coded by structural equation type. Three things are consistent across datasets and methods: (i) linear (poly-deg1) equations form a tight cluster; (ii) transcendental classes (trig, log) occupy distinct regions; (iii) PCA captures the same broad structure as t-SNE, indicating the family-level separation is not a t-SNE artifact.

.8. Ablations

We summarize the ablation findings; the corresponding learning curves are in Figures 8 and 7.

- Replacing dense complexity-delta rewards with sparse outcome-only rewards (and disabling the inverse-frequency curriculum) substantially degrades performance. The main drawback of sparse rewards is *far* longer run times: we had to introduce a 1-second-per-step wall-clock timeout to keep training tractable.
- TreeMLP outperforms a vanilla GCN and GraphSAGE on the small dataset (Figure 7); the tree-aware aggregation appears necessary, not just helpful.
- Curiosity bonuses (ICM, RND, NGU) do not improve TreeMLP learning on `abel_level13` (Figure 9). At their full 3×10^6 -step horizon, ICM and NGU roughly track the no-curiosity baseline on all four metrics (coverage, greedy, beam, success@10); RND produces a near-flat learning curve, failing to make meaningful progress. This contrasts with earlier work (Dabelow & Ueda, 2024), where the same family of bonuses *improved* a vanilla MLP policy on a closely related

task. A plausible explanation: the TreeMLP’s structural inductive bias already supplies the exploration signal that curiosity provides for the MLP, making them redundant or—in the case of RND—actively conflicting.

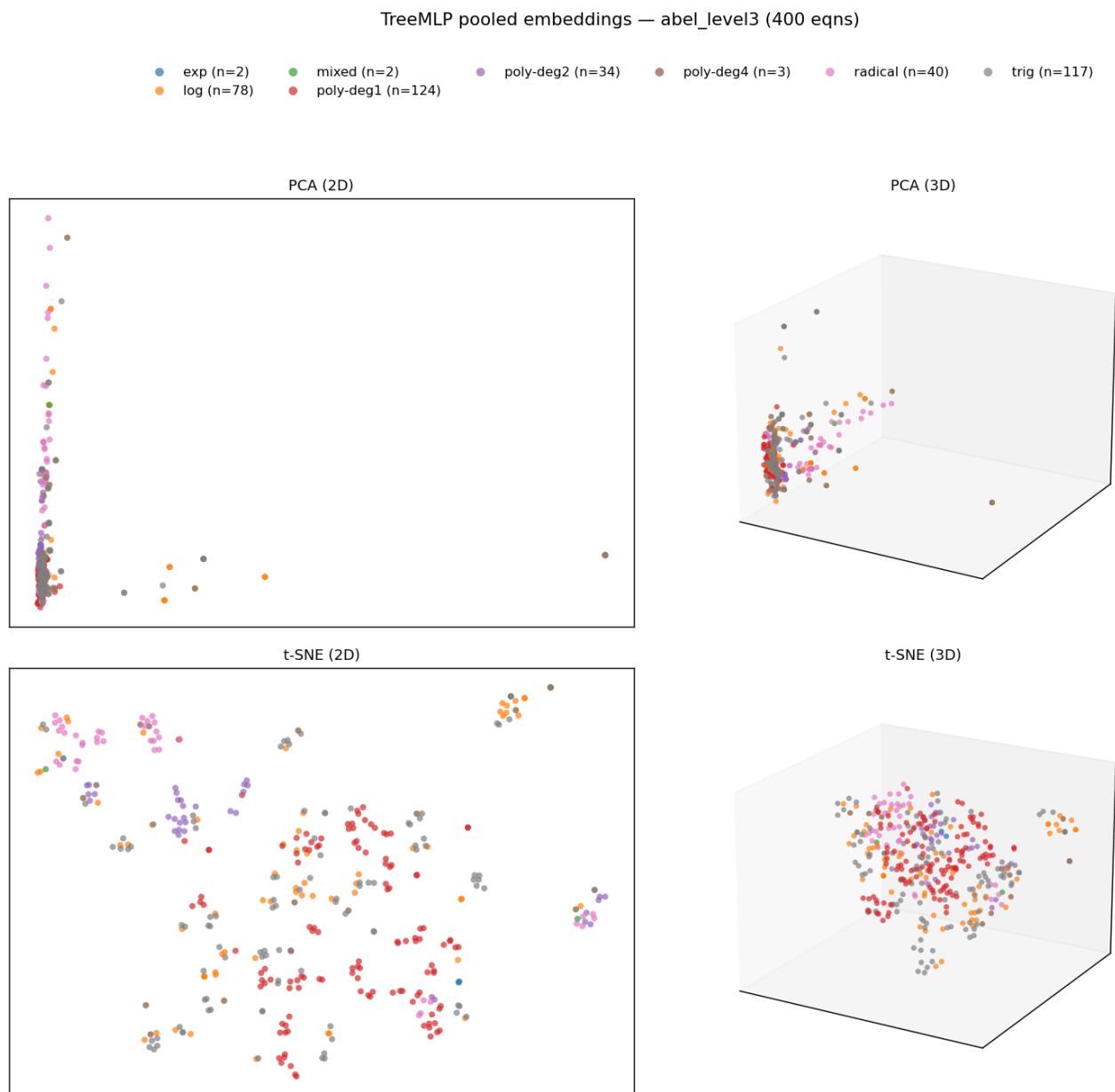


Figure 4. TreeMLP embeddings on the small dataset, projected by PCA (top) and t-SNE (bottom), in 2D (left) and 3D (right). 400 random equations, colored by structural type. The same cluster structure is visible across all four projections.

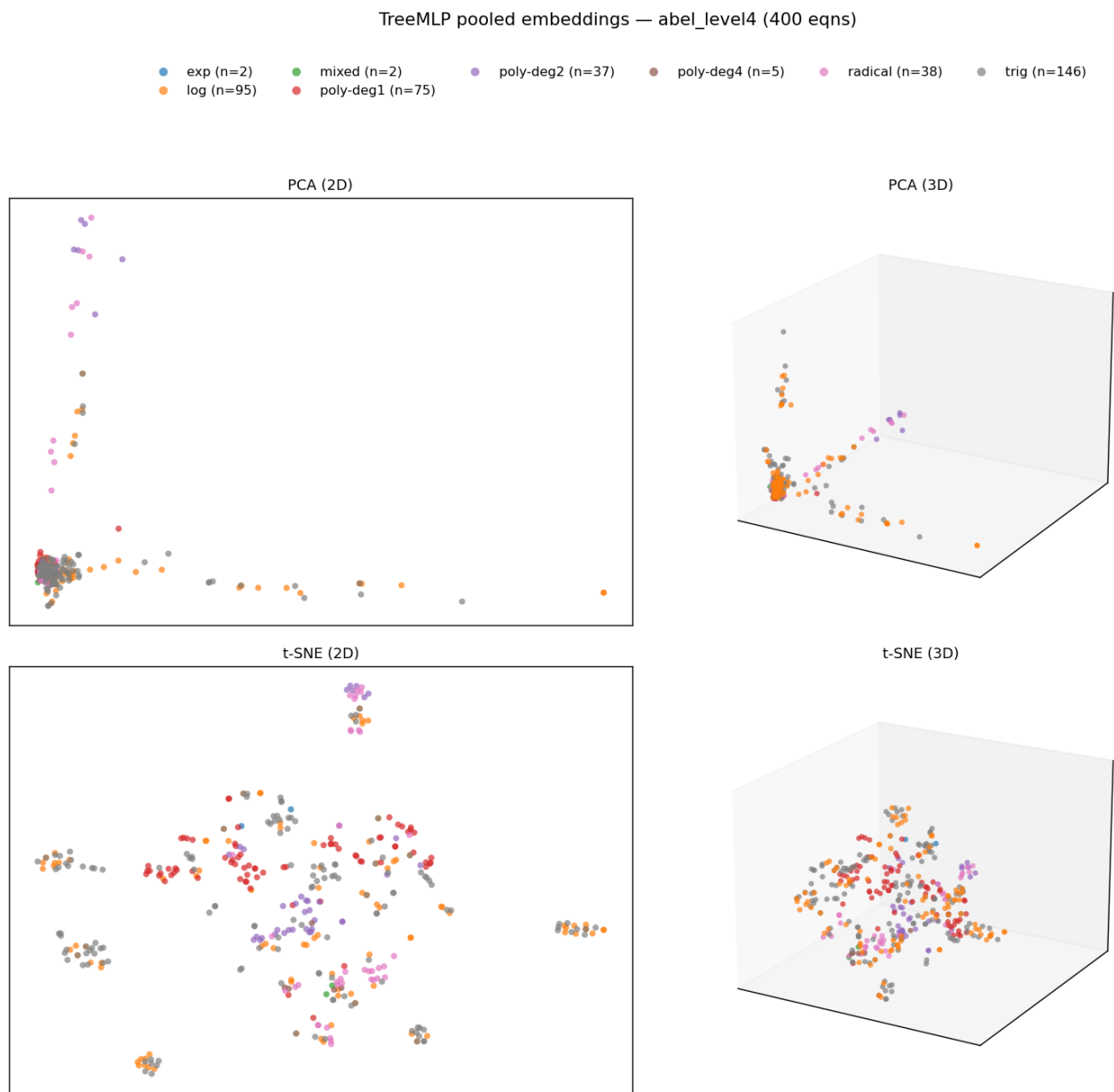


Figure 5. Same as Figure 4 but on the large dataset (depth < 5). More types are present and cluster boundaries are softer, consistent with the lower per-type accuracies reported in the main text.

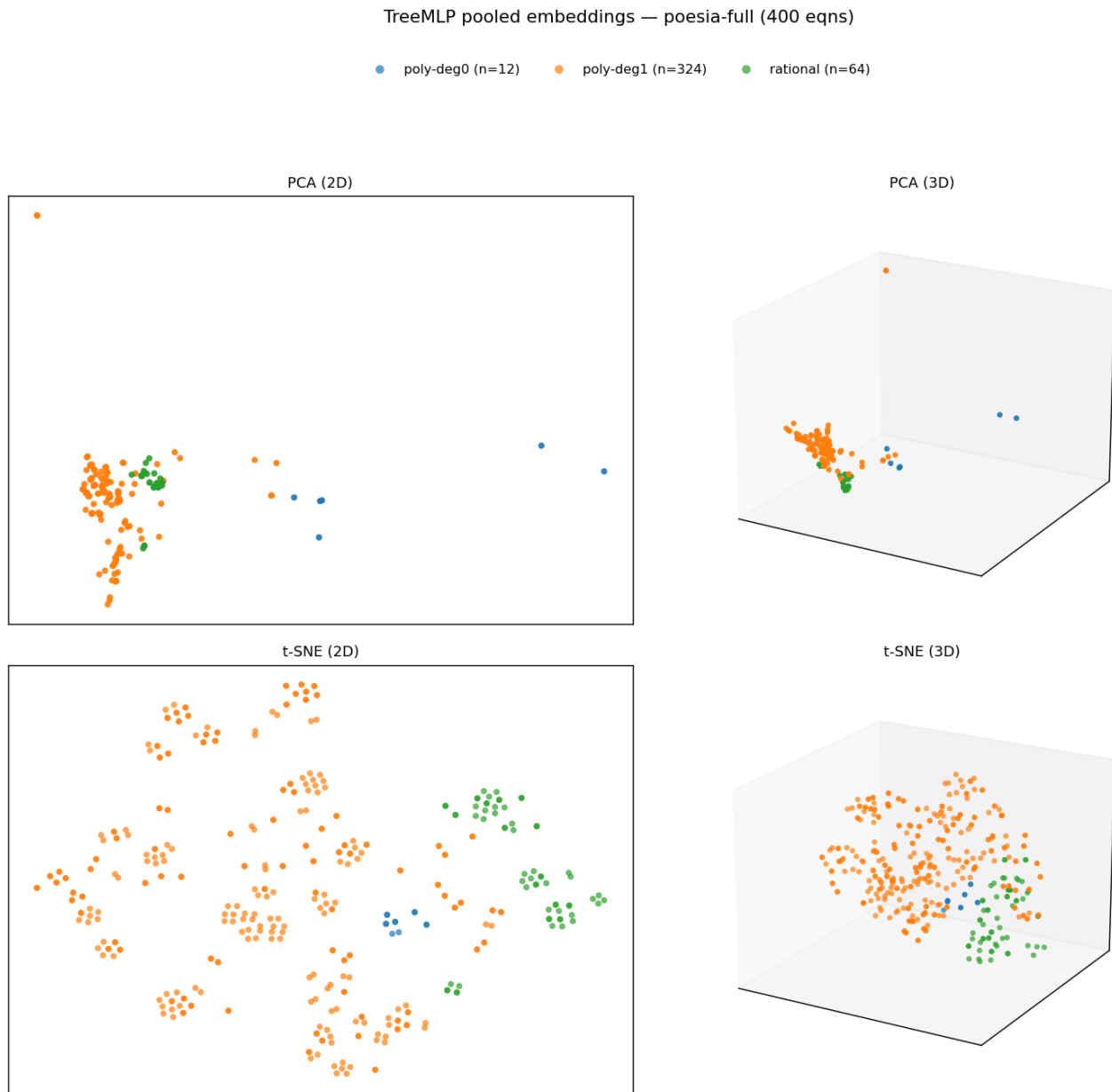


Figure 6. Same as Figure 4 but on CommonCore (poesia). The dataset is dominated by linear and rational equations, giving a simpler embedding geometry.

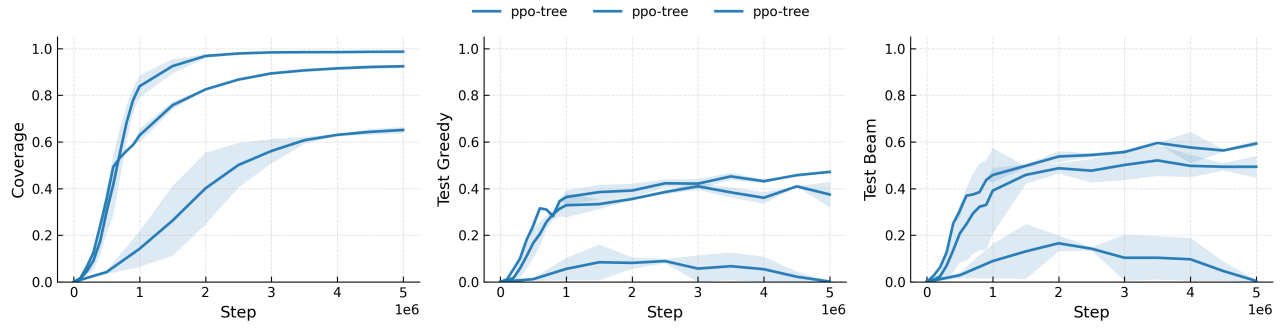


Figure 7. TreeMLP outperforms GCN and GraphSAGE on the small datasets. Mean of three random seeds with shading to min/max.

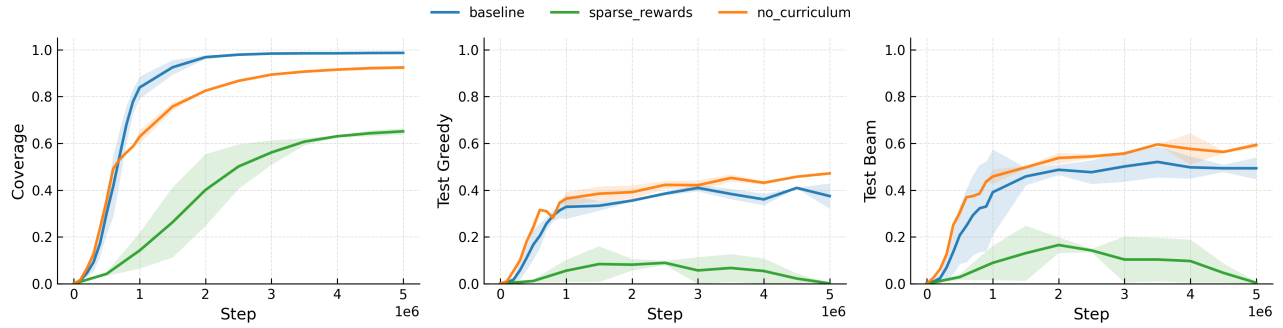


Figure 8. Ablation study on small datasets.

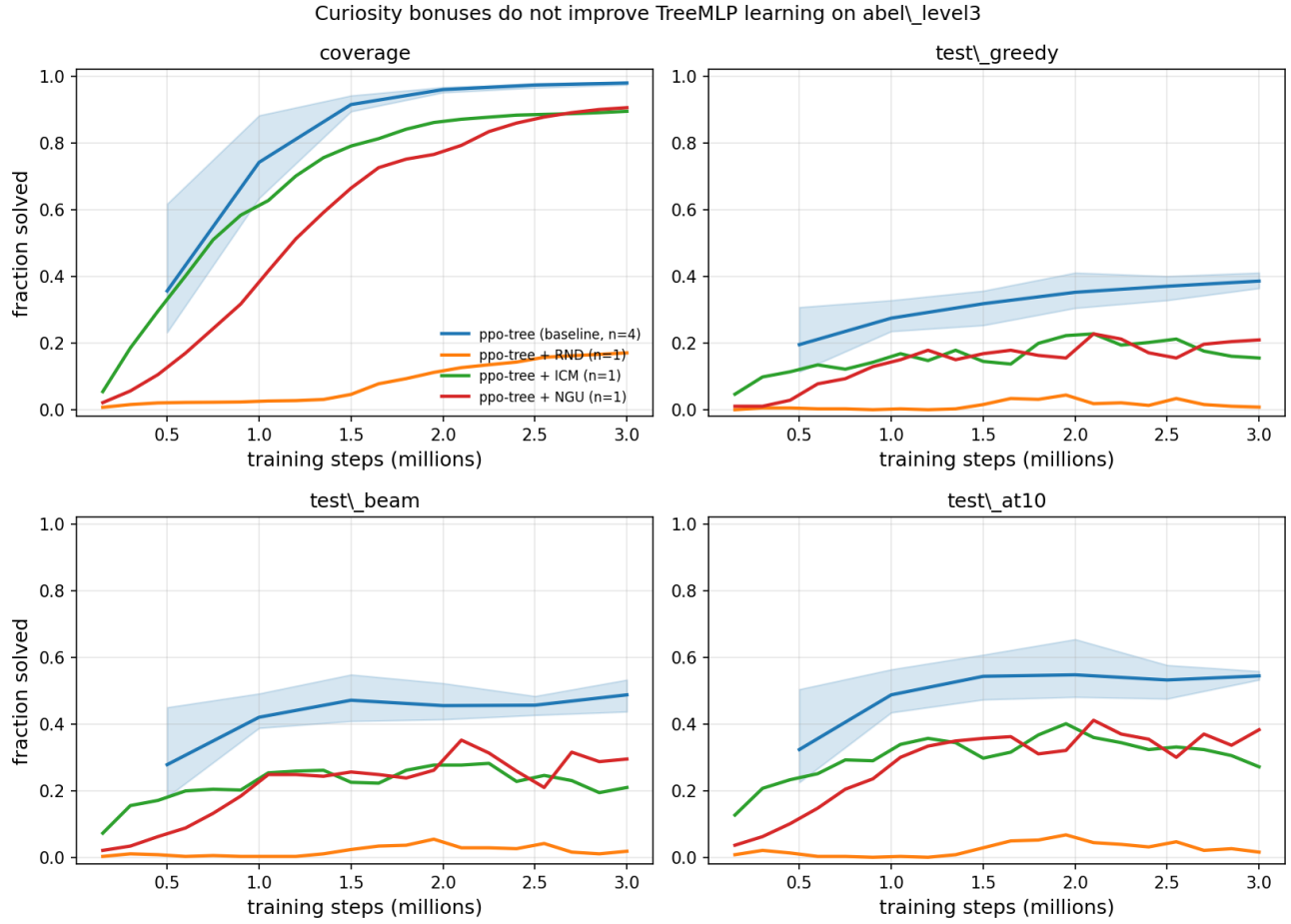


Figure 9. Curiosity bonuses (ICM, NGU, RND) do not improve TreeMLP learning on abel_level3. At 3×10^6 steps, ICM and NGU roughly track the no-curiosity baseline; RND produces a near-flat curve. Baseline is mean $\pm$ min/max across $n=5$ seeds; curiosity variants are single seed. The previous version of this figure used a 5×10^5 horizon at which the baseline itself had not yet risen; the difference is now clearly visible.